

How Much Scalability Do You Have or Need?

"Orders" of Throughput

By Herb Sutter

Herb is a software architect at Microsoft and chair of the ISO C++ Standards committee. He can be contacted at www.gotw.ca.

In your application, how many independent pieces of work are ready to run at any given time? Put another way, how many cores (or hardware threads, or nodes) can your code harness to get its answers faster? And when should the answers to these questions not be "as many as possible"?

Table 1 summarizes three "orders" of throughput available in a given application or operation, using big-Oh notation to describe the amount of CPU-intensive work that will typically be ready to be actively executed at a given time. The rest of this article overviews these orders of scalability and summarizes when each one is appropriate.

Last month [2], we looked at three "pillars" of concurrency: Pillar 1, isolation by structuring work asynchronously and communicating through messages; Pillar 2, scalable throughput by exploiting parallelism in algorithms and data structures; and Pillar 3, dealing with mutable shared state. This month's topic delves into throughput-oriented techniques using techniques in Pillar 1 and Pillar 2.

Order	O(1): Single-Core	O(K): Fixed	O(N): Scalable
Tagline	One thing at a time	Explicit threading	Re-enable the free lunch
Summary	Sequential applications, and bottlenecked parallel applications	Explicitly express how much work can be done in parallel	Express lots of latent concurrency in a way that can be efficiently mapped to N cores
Examples	Multithreaded code convoyed on a global lock or message queue, occasional or intermittent background work	Pipelining, hardwired division of tasks, regular or continuous background computation	Tree traversal, quicksort, compilation
Applicability	Single-core hardware, single-threaded OS, or nonCPU-bound app	Hardware with fixed concurrency, or app whose CPU-bound parts have limited scalability	Hardware with variable (esp. growing) concurrency, and app with CPU-bound parts that are scalably parallelizable
Examples	Code targeting legacy hardware, small embedded systems, single-core game consoles; simple text processor	Game targeting one multicore game console generation; code whose key operations are order-sensitive (e.g., can be pipelined but not fully parallelized)	Mainstream desktop or server software with CPU-bound features and targeting commodity hardware or future upgradeable game consoles
Pillar (see Notes [2]), and today's mainstream tools	Pillar 1: Threads, message queues, futures		Pillar 2: Thread pools, futures, OpenMP

Table 1: Orders of throughput.

O(1): Sequential Code

O(1) means that the program typically has one CPU-intensive piece of work available to be actively executed at any given time. It may occasionally

perform multiple concurrent operations, such as occasional background work in addition to the main foreground work, but the extra work is not ongoing and/or doesn't keep more than a single core busy.

This category includes not only all sequential code, but also every concurrent application with throughput that is as good as sequential because its threads execute serially, such as by being convoyed on a global lock or message queue. The free throughput lunch is over for both these kinds of $O(1)$ code, but the fully serial option tends to have the additional liability of poorer responsiveness, while a concurrent application tends to be better structured to do background work asynchronously[1, 2].

For example, consider Microsoft Word: As I type this paragraph, WINWORD.EXE has between 8 and 11 active threads, but all CPU cores remain virtually idle. Each time Word regularly performs a background Save, one core's utilization increases to more than 50 percent for about two seconds before the system returns to idle. This version of Word is $O(1)$ when used in this mode; I'm not likely to see improvement when running it on a system with a greater number of hardware cores, even though the application uses several threads internally.

In all $O(1)$ cases, if we want better throughput for CPU-bound operations, essentially, our only option is to try to optimize our code in traditional ways because adding more cores won't help.

$O(K)$: Explicitly Threaded Code

$O(K)$ means that the system typically has some constant number of things it can do to keep a constant number of cores busy at any given time. That number is hardwired into the structure of the program and will be the same regardless of the amount of hardware concurrency available at execution time.

For example, consider a first-person action game. To take some advantage of additional cores, let's say we divide the game's compute-bound work into three threads: Thread 1 does game physics, thread 2 does rendering, and thread 3 does nonplayer character AI. For simplicity, assume that all three threads are equally busy and interdependent. The game runs fine on a single-core machine; the operating system just interleaves the threads on the one core. When the user upgrades to a two-core machine, the game runs faster—but not twice as fast, because if we schedule thread 1 on core 1, and thread 2 on core 2, we have to put thread 3 somewhere. If we put thread 3 on the same core as thread 1, then thread 2 (which depends on 1 and/or 3) will be idle half the time. But if we schedule thread 3 on both core 1 and core 2, we incur cache sloshing and other overhead every time we move it from one core to the other. So the game runs faster on a two-core system, just not twice as fast. When the user upgrades to a four-core machine, the game runs faster still—but only three times as fast as on a single core, or maybe slightly better if spyware and other applications can be moved to the fourth core. When the user upgrades to an eight-core machine, nothing happens. When he upgrades to a 16-core machine, more nothing happens. The $O(3)$ application is hardwired to prefer three cores regardless of the input or execution environment.

A closely related technique is pipelining, which is useful when we have a collection of things to operate on, but we can't apply $O(N)$ techniques to fully parallelize them because of ordering dependencies. For example, consider a communications subsystem that packets data to be sent over the network. The input to the subsystem is a stream of raw packets; before sending a given packet, it must first be decorated with header information, then compressed, and then encrypted. Those three operations can't easily be run in parallel for a given packet, in part because they must be applied in that order. A solution is to have three agents that communicate using message queues: The *Decorator* agent (typically a thread) decorates each raw packet and sends it to the *Compressor* agent; the *Compressor* accepts the decorated packet, compresses it, and sends it to the *Encryptor*; and the *Encryptor* encrypts it and sends it to the network. Like the game example, this subsystem also has $O(3)$ parallelism if the pipeline stages are equally busy. We are using Pillar 1 techniques (isolated agents, messaging; see [2] for details) because it's a natural fit for expressing pipelines: The pipeline stages are relatively isolated, and using messaging lets us tolerate any difference in execution rates between the stages.

Finally, consider Word again, but with a different usage scenario: Dictation. As I fire up speech recognition and dictate this paragraph, I see that generally two cores stay fairly busy as I'm speaking, while the rest of the system's cores remain idle. When used in dictation mode, this version of Word is an $O(2)$ application that works fine on a single-core machine, better on a dual-core machine, but won't improve further in speed or accuracy in the presence of additional cores.

An $O(K)$ application is explicitly structured to prefer K cores for a given input workload. $O(K)$ code can't scale to take advantage of environments with more than K cores, and it can penalize execution environments with fewer than K cores by interfering with load balancing and causing some of the cores to be only partly utilized.

Comparing $O(1)$ and $O(K)$

Alert readers may already have noticed that in other computer science contexts we usually don't distinguish between $O(1)$ and $O(K)$ complexity because big-Oh notation typically ignores the constant factor; after all, $K=1$ is just a special case. That's exactly true, and similarly here $O(K)$ is far closer to $O(1)$ than it is to $O(N)$. Both $O(1)$ and $O(K)$ hardwire concurrency into the explicit structure of the code, and neither is scalable to arbitrary numbers of cores. Even so, it's well worth distinguishing between $O(1)$ and $O(K)$.

$O(1)$ is an important special case because it targets three significant situations:

- Single-core hardware, including legacy hardware and some game consoles, such as Nintendo Wii.
- Operating systems that don't support concurrency, including some that have no concept of a thread.
- Applications that aren't CPU-bound, where their current feature set won't drive more than one core's worth of computation no matter how you slice them (a simple text processor, for instance).

If you have no reason or capability to escape one of those constraints, it probably doesn't make sense to invest heavily in making your code $O(K)$ or better because the extra concurrency will never be exercised on such systems. But don't forget that, even in the $O(1)$ world, concurrency is a primary path to better responsiveness, and some basic techniques like using an event-driven program structure will work even in the most constrained case of running on an OS that has no support for actual threads. $O(K)$ is appropriate especially for code that targets domains with fixed concurrency:

- Fixed hardware targets: A notable situation is when you're targeting one multicore game console generation. For example, when developing a game for Xbox 360, it makes sense to write an $O(6)$ application, because that generation of the console is fixed at six hardware threads (three cores with two hardware threads each); similarly, PlayStation 3 naturally lends itself to $O(8)$ or $O(9)$ applications because it is fixed at 1+8 cores (one general-purpose core, and eight special-purpose cores). These configurations will not change until each console's next major hardware generation.
- Applications whose key operations cannot be fully parallelized: The earlier packet-sending example illustrates a case where a CPU-intensive operation has internal ordering dependencies; its parts are not sufficiently independent to let them be run fully in parallel. As we saw, pipelining is a classic approach to extract some parallelism out of an otherwise inherently serial operation; but basic pipelining is at best $O(K)$, for a pipeline with K stages of relatively equal cost. Although $O(K)$ is not scalable, it can be deployed tactically to temporarily extend the free lunch.

In $O(K)$ in particular, there are some fixed costs of making the code concurrent that we will incur at runtime on even a single-core machine, including work to manage messages, perform context switches, and lock shared data. However, by applying concurrency, we can get some limited scalability, and often also better responsiveness.

Note that $O(1)$ and $O(K)$ situations are mostly described by what you can't do. If you can target hardware that has variable parallelism (especially increasing parallelism), and your operating system supports concurrency, and you can find enough independence inside your application's CPU-intensive operations to make them scale, prefer aiming for $O(N)$.

$O(N)$: Scalable Throughput And the Free Lunch

The key to scalable throughput is to express lots of latent concurrency that isn't explicitly coded in the program and that scales to match its inputs (number of messages, size of data, and so on) and that can be efficiently mapped down at execution time to the variable number N cores available on a given machine.

We find the scalable concurrency opportunities principally by exploiting natural parallelism:

- Exploit parallelism in algorithm structures: For example, recursive sorting can exploit the natural parallelism in its divide-and-conquer structure.
- Exploit parallelism in data structures: For example, tree traversal can often exploit the independence in each node's subtrees. Compilation can exploit independence at several levels in the structure of source code, from coarse-grained independence among source files to finer-grained independence among classes or methods within a file.

This lets us decompose the application into a "sea of chores"—expressing independent chunks of work that are big or small, blocking or nonblocking, structured subdivisions or independent; but we often want to look first for the CPU-bound work—and rely on the runtime system to "rightsize" the application by assigning those chores efficiently to whatever hardware parallelism is available on a given user's system.

OpenMP supports some constrained $O(N)$ styles, but it is primarily intended for use with integer-indexed loops over arrays and doesn't work well with iteration-based containers in STL, .NET, or Java. Instead, as mentioned in last month's column [2], today, one common idiom for expressing such a sea of work items is to explicitly schedule chores for execution on a thread pool (for example, using Java *ThreadPoolExecutor* or .NET *BackgroundWorker*). Unfortunately, this incurs significant context-switch overhead and so you have to ensure that a work item will be worth shipping over to a worker thread. In the future, this constraint will be relaxed as languages and runtimes improve.

$O(N)$ is the key to re-enabling the free lunch and keeping lots of cores busy, because it lets us express applications that can run on yesterday's single-core machine, run better on today's four-core machine, run better still on tomorrow's 64-core machine, and so on until we exhaust the inherent limit of CPU-boundedness in the application. For a thread pool-driven program, on a single-core machine the system can spin up a single worker thread that runs the program by serially pulling chore after chore from the sea and executing it; on an eight-core machine, it can spin up eight threads; and so on [4].

As with $O(K)$, $O(N)$ can have costs at runtime on even a single-core machine. In addition to the costs mentioned for $O(K)$, there can be extra work inherent in divide-and-conquer techniques (e.g., reductions such as piecing together a grand total from intermediate results), and the cost of locking shared data can now also increase. However, by applying concurrency, we can often get good scalability that far offsets the overhead.

Using $O(N)$ is highly desirable for software that expects to run on a variety of current and future hardware having variable amounts of hardware parallelism—which happens to now be the target for all mainstream desktop and server software. If your application doesn't currently have key CPU-bound operations that are amenable to full $O(N)$ parallelization, don't give up: Consider finding new desirable features that are amenable to $O(N)$, or at least more $O(K)$, and you will still be able to deliver software that runs well on today's hardware, better on tomorrow's hardware, and better still on future systems.

Notes

[1] I use "cores" as a simple shorthand measure of execution hardware parallelism. For applications running on one local machine the appropriate measure is usually "total hardware threads," meaning $\#sockets \times \#cores/socket \times \#threads/core$; and for distributed applications the appropriate measure is usually "nodes."

[2] H. Sutter. "The Pillars of Concurrency" (*DDJ*, 32(7), August 2007, <http://ddj.com/dept/64bit/200001985>).

[3] H. Sutter. "The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software" (*DDJ*, 30(3), March 2005, <http://ddj.com/dept/webservices/184405990>).

[4] The details are more complex. For example, a pool should spin up extra worker threads to keep the system busy whenever one chore blocks to wait for some event and so temporarily idles its worker thread.